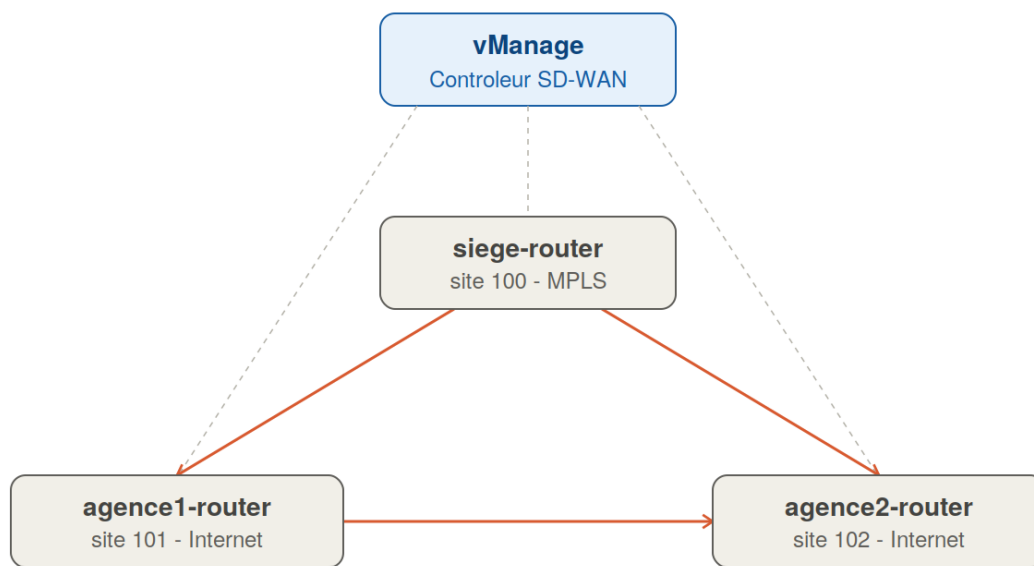


TP19 — Configurer une topologie SD-WAN

Formation NetOps : Automatiser la gestion de son Infrastructure Réseau — Ambient IT

Description



Pointille = plan de gestion (vManage) -- Plein = overlay OMP entre sites

Figure 1: Topologie SD-WAN à configurer

Cisco SD-WAN (Viptela) s'appuie sur un contrôleur central, vManage, dont l'authentification suit un cycle en deux temps encore différent des TP précédents : un login par formulaire qui ouvre une session (cookie), puis la récupération d'un jeton XSRF obligatoire pour toute action modifiant la configuration. Ce TP simule ce cycle, configure une topologie SD-WAN à 3 sites (identifiants de site, transport TLOC), et vérifie que l'overlay se forme réellement (sessions OMP entre sites).

Prérequis

- TP1 à TP18 validés

Étapes

Étape 1 — Créer sa branche

```
cd netops-project-<nom-groupe>
git checkout dev
git pull origin dev
git checkout -b <prenom>-<nom>-jour5-tp19
```

Étape 2 — Créer le mock vManage

```
cat > scripts/mock_vmanage_api.py << 'EOF'
import secrets
```

```
from flask import Flask, request, jsonify, session
```

```
app = Flask(__name__)
app.secret_key = "dev-secret-key-not-for-production"
```

```
VALID_USER = "admin"
VALID_PASS = "vmanagepass123"
```

```
DEVICES = [
    {"deviceId": "vedge-siege", "host-name": "siege-router", "site-id": None, "rea
    {"deviceId": "vedge-agence1", "host-name": "agence1-router", "site-id": None,
    {"deviceId": "vedge-agence2", "host-name": "agence2-router", "site-id": None,
]
```

```
XSRF_TOKENS = {}
```

```
def find_device(device_id):
    return next((d for d in DEVICES if d["deviceId"] == device_id), None)
```

```
def check_session():
    return session.get("authenticated") is True
```

```
def check_xsrp():
    token = request.headers.get("X-XSRF-TOKEN")
    return token and XSRF_TOKENS.get(session.get("sid")) == token
```

```
@app.route("/j_security_check", methods=["POST"])
def j_security_check():
    username = request.form.get("j_username")
    password = request.form.get("j_password")
```

```

    if username != VALID_USER or password != VALID_PASS:
        return "Login failed", 401
    session["authenticated"] = True
    session["sid"] = secrets.token_hex(8)
    return "", 200

@app.route("/dataservice/client/token", methods=["GET"])
def client_token():
    if not check_session():
        return jsonify({"error": "Not authenticated"}), 401
    token = secrets.token_hex(16)
    XSRF_TOKENS[session["sid"]] = token
    return token, 200

@app.route("/dataservice/system/device/vedges", methods=["GET"])
def list_vedges():
    if not check_session() or not check_xsrf():
        return jsonify({"error": "Missing or invalid session/XSRF token"}), 401
    return jsonify({"data": DEVICES})

@app.route("/dataservice/system/device/site", methods=["POST"])
def configure_site():
    if not check_session() or not check_xsrf():
        return jsonify({"error": "Missing or invalid session/XSRF token"}), 401
    body = request.get_json(force=True)
    device = find_device(body.get("deviceId"))
    if not device:
        return jsonify({"error": f"Device inconnu : {body.get('deviceId')}"}), 404
    device["site-id"] = body.get("siteId")
    device["tloc-color"] = body.get("tlocColor")
    return jsonify({"status": "success", "device": device})

@app.route("/dataservice/device/omp/summary", methods=["GET"])
def omp_summary():
    if not check_session() or not check_xsrf():
        return jsonify({"error": "Missing or invalid session/XSRF token"}), 401
    device_id = request.args.get("deviceId")
    device = find_device(device_id)
    if not device:
        return jsonify({"error": f"Device inconnu : {device_id}"}), 404
    if not device["site-id"]:
        return jsonify({"data": {"omp-peers-up": 0, "status": "site non configure"}})

    configured_others = [d for d in DEVICES if d["site-id"] and d["deviceId"] != device_id]

```

```

        return jsonify({"data": {"omp-peers-up": len(configured_others), "status": "up"})

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=8000, debug=False)
EOF

```

Trois mécanismes d'authentification différents en 3 TP : X-Auth-Token (DNA Center, TP15), APIC-Cookie (ACI, TP17), et ici une session cookie + jeton XSRF séparé (vManage) — chaque plateforme Cisco a ses propres conventions.

Étape 3 — Créer le client vManage

```

cat > scripts/netops_client/vmanage_client.py << 'EOF'
import requests

class VManageError(Exception):
    """Erreur levee lors d'un echange avec l'API vManage (SD-WAN)."""
    pass

class VManageClient:
    """Client minimal pour l'API Cisco SD-WAN vManage : authentication par session"""

    def __init__(self, base_url, username, password, timeout=5):
        self.base_url = base_url.rstrip("/")
        self.username = username
        self.password = password
        self.timeout = timeout
        self.session = requests.Session()
        self.xsrf_token = None

    def login(self):
        url = f"{self.base_url}/j_security_check"
        response = self.session.post(
            url,
            data={"j_username": self.username, "j_password": self.password},
            timeout=self.timeout,
        )
        if response.status_code != 200:
            raise VManageError(f"Authentication refusee ({response.status_code})")

        token_response = self.session.get(f"{self.base_url}/dataservice/client/token")
        if token_response.status_code != 200:
            raise VManageError("Impossible de recuperer le jeton XSRF")
        self.xsrf_token = token_response.text

```

```

def _headers(self):
    if not self.xsrf_token:
        raise VManageError("Non authentifie : appelez login() d'abord")
    return {"X-XSRF-TOKEN": self.xsrf_token}

def get_vedges(self):
    url = f"{self.base_url}/dataservice/system/device/vedges"
    response = self.session.get(url, headers=self._headers(), timeout=self.timeo
    if response.status_code != 200:
        raise VManageError(f"Erreur inventaire vEdges : {response.text}")
    return response.json()["data"]

def configure_site(self, device_id, site_id, tloc_color):
    url = f"{self.base_url}/dataservice/system/device/site"
    body = {"deviceId": device_id, "siteId": site_id, "tlocColor": tloc_color}
    response = self.session.post(url, headers=self._headers(), json=body, timeo
    if response.status_code != 200:
        raise VManageError(f"Echec de la configuration du site : {response.text}")
    return response.json()

def omp_summary(self, device_id):
    url = f"{self.base_url}/dataservice/device/omp/summary"
    response = self.session.get(url, headers=self._headers(), params={"deviceId
    if response.status_code != 200:
        raise VManageError(f"Erreur OMP summary : {response.text}")
    return response.json()["data"]
EOF

cat > scripts/netops_client/__init__.py << 'EOF'
from .vmanage_client import VManageClient, VManageError

__all__ = ["VManageClient", "VManageError"]
EOF

```

Étape 4 — Créer le script de configuration de la topologie

```

cat > scripts/configure_sdwan_topology.py << 'EOF'
import os
import sys

from netops_client import VManageClient, VManageError

VMANAGE_URL = os.environ.get("VMANAGE_URL", "http://localhost:8000")
VMANAGE_USER = os.environ.get("VMANAGE_USER", "admin")
VMANAGE_PASS = os.environ.get("VMANAGE_PASS")

if not VMANAGE_PASS:

```

```

    print("ERREUR : variable d'environnement VMANAGE_PASS non definie.")
    sys.exit(1)

SITE_TOPOLOGY = [
    ("vedge-siege", 100, "mpls"),
    ("vedge-agence1", 101, "biz-internet"),
    ("vedge-agence2", 102, "biz-internet"),
]

def main():
    client = VManageClient(base_url=VMANAGE_URL, username=VMANAGE_USER, password=VMANAGE_PASS)
    client.login()
    print("Authentifie sur vManage.\n")

    devices = client.get_vedges()
    print(f"{len(devices)} vEdge(s) trouve(s) dans l'inventaire :")
    for d in devices:
        print(f"  - {d['host-name']} ({d['deviceId']}) : {d['reachability']}")
    print()

    print("=== Configuration de la topologie SD-WAN ===")
    for device_id, site_id, tloc_color in SITE_TOPOLOGY:
        client.configure_site(device_id, site_id, tloc_color)
        print(f"  {device_id} -> site-id {site_id}, TL0C {tloc_color}")
    print()

    print("=== Verification de l'overlay (sessions OMP) ===")
    all_up = True
    for d in devices:
        summary = client.omp_summary(d["deviceId"])
        print(f"  {d['host-name']} ({d['deviceId']}) : {summary['omp-peers-up']} pairs")
        if summary["status"] != "up":
            all_up = False

    if not all_up:
        print("\nATTENTION : au moins un site n'a pas etabli de session OMP avec le cloud")
        sys.exit(1)

    print("\nTopologie SD-WAN entierement formee (tous les sites se voient via OMP)")

if __name__ == "__main__":
    try:
        main()
    except VManageError as e:
        print(f"Erreur vManage : {e}")
        sys.exit(1)

```

EOF

Notez la vérification finale : elle boucle sur tous les appareils de l'inventaire (devices), pas seulement sur ceux qu'on vient de configurer dans SITE_TOPOLOGY — c'est ce qui permet de détecter un site qu'on aurait oublié d'ajouter à la topologie, plutôt que de se contenter de vérifier ce qu'on a fait.

Étape 5 — Tester la configuration complète

```
nohup python scripts/mock_vmanage_api.py > /tmp/vmanage.log 2>&1 &
sleep 2
export VMANAGE_PASS=vmanagepass123
python scripts/configure_sdwan_topology.py
```

Résultat attendu :

Authentifie sur vManage.

```
3 vEdge(s) trouve(s) dans l'inventaire :
- siege-router (vedge-siege) : reachable
- agence1-router (vedge-agence1) : reachable
- agence2-router (vedge-agence2) : reachable
```

```
=== Configuration de la topologie SD-WAN ===
vedge-siege -> site-id 100, TLOC mpls
vedge-agence1 -> site-id 101, TLOC biz-internet
vedge-agence2 -> site-id 102, TLOC biz-internet
```

```
=== Verification de l'overlay (sessions OMP) ===
siege-router (vedge-siege) : 2 pair(s) OMP actif(s), statut = up
agence1-router (vedge-agence1) : 2 pair(s) OMP actif(s), statut = up
agence2-router (vedge-agence2) : 2 pair(s) OMP actif(s), statut = up
```

Topologie SD-WAN entièrement formée (tous les sites se voient via OMP).

Étape 6 — Simuler un site oublié dans la topologie

Retirez temporairement la ligne ("vedge-agence2", 102, "biz-internet"), de SITE_TOPOLOGY dans le script.

Redémarrez impérativement le mock avant de retester : l'état de chaque site (site-id) vit en mémoire dans le processus Flask. Si le mock tourne déjà depuis un test précédent, agence2 reste configuré tel qu'il l'était, peu importe ce que vous commentez dans le script — commenter une ligne dit seulement "ne pas reconfigurer cette fois", pas "oublier ce qui a déjà été fait".

```
netstat -ano | grep ":8000" | grep LISTENING | awk '{print $5}' | sort -u | wh
nohup python scripts/mock_vmanage_api.py > /tmp/vmanage.log 2>&1 &
```

```
sleep 2
```

Relancez :

```
python scripts/configure_sdwan_topology.py
```

Résultat attendu :

```
=== Verification de l'overlay (sessions OMP) ===
siege-router (vedge-siege) : 1 pair(s) OMP actif(s), statut = up
agence1-router (vedge-agence1) : 1 pair(s) OMP actif(s), statut = up
agence2-router (vedge-agence2) : 0 pair(s) OMP actif(s), statut = site non configure
```

ATTENTION : au moins un site n'a pas établi de session OMP avec le reste de l'overlay.

Étape 7 — Remettre la ligne oubliée

Restaurez ("vedge-agence2", 102, "biz-internet"), dans SITE_TOPOLOGY, puis redémarrez le mock une dernière fois pour repartir sur un état propre avant de committer.

Étape 8 — Committer et pousser

```
git add scripts/mock_vmanage_api.py scripts/netops_client/vmanage_client.py scripts/
git commit -m "TP19 : configuration d'une topologie SD-WAN (site-id, TLOC, verification)"
git push -u origin <prenom>-<nom>-jour5-tp19
```

Ouvrir une pull request <prenom>-<nom>-jour5-tp19 → dev.

Comprendre le TP

Le cycle d'authentification vManage est le plus élaboré des trois vus jusqu'ici : un login par formulaire (j_security_check) établit une session, puis un second appel récupère un jeton XSRF à fournir sur chaque requête suivante. Cette double protection reflète le fait que vManage est aussi une interface web complète, pas seulement une API — le jeton XSRF protège contre des requêtes malveillantes qui utiliseraient la session d'un administrateur déjà connecté.

site-id et TLOC color sont les deux informations qui font qu'un routeur rejoint réellement l'overlay SD-WAN : le site-id identifie logiquement l'emplacement physique, et la couleur TLOC (mpls, biz-internet) identifie le type de transport sous-jacent — c'est cette couleur que les politiques peuvent utiliser pour orienter le trafic.

OMP (Overlay Management Protocol) est le protocole par lequel les vEdges annoncent leurs routes et se découvrent mutuellement, un peu comme un BGP dédié au SD-WAN. Vérifier le nombre de pairs OMP actifs après configuration est le seul moyen de confirmer que la topologie fonctionne réellement — comme pour l'ACI (TP17), un déploiement technique réussi ne garantit pas encore que le résultat métier (l'overlay complet) est atteint.

Validation

- ☐ `python scripts/configure_sdwan_topology.py` s'authentifie (session + XSRF), liste 3 vEdges, configure les 3 sites
- ☐ La vérification OMP montre 2 pair(s) actif(s) et statut = up sur chaque site en temps normal
- ☐ Un site oublié dans `SITE_TOPOLOGY` (après redémarrage du mock) apparaît avec 0 pair(s) et statut = site non configure, et fait échouer le script
- ☐ Une pull request <prenom>-<nom>-jour5-tp19 → dev est ouverte